

Change Detection

Change detection is how Angular decides **when to re-check a template** and update the DOM. By default it checks every component every time; **OnPush** checks only what might have changed. This guide covers Zone.js, the two strategies, the mutate-vs-replace trap, signals, the tools, and how to make an app faster.

1 · WHAT IT IS

How Angular keeps the screen in sync

Change detection is the process that compares what your templates *show* with your component data and updates the DOM when they differ. It runs after something might have changed:

- a user event (click, input, keypress)
- a timer (`setTimeout`, `setInterval`)
- a finished request (HTTP, promises)

Angular learns about these through **Zone.js**, a library that wraps the browser APIs above. After each one it checks the component tree **top-down**, once.

Something happens → Zone.js notices → Angular checks the tree → DOM updated if values changed

TIP

Change detection checks **bindings in templates** (`{{ name }}`, `[disabled]`, `@if ...`). It does not "watch" your variables directly.

2 · TWO STRATEGIES

Default checks everything, OnPush checks only what might have changed

	DEFAULT	ONPUSH
Checked when	Every time anything in the app is checked	Only when it is marked dirty (list below)
Cost	Simple, but wasteful in big trees	Much less work; skipped subtrees cost nothing
How to enable	Nothing (it is the default)	<code>changeDetection:</code> <code>ChangeDetectionStrategy.OnPush</code>

An OnPush component is re-checked when...

- one of its `input()`s receives a **new reference** (not a mutated one),
- an **event** fires from its template or one of its children (a click inside it),
- an `async` pipe in its template emits,
- a **signal** read in its template changes,
- you call `markForCheck()` (or `detectChanges()`) yourself.

```
@Component({
  selector: 'app-user-card',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `{{ user().name }}`,
})
export class UserCardComponent {
  readonly user = input.required<User>();
}
```

3 · THE CLASSIC TRAP

Mutate vs. replace

OnPush compares inputs by **reference** (`===`). If you change a property of the same object, the reference is identical, so Angular sees "no change" and skips the component.

Mutating — OnPush does not update

```
user.name = 'Grace';
items.push(newItem);
filters.active = true;
```

Replacing — OnPush updates

```
user = { ...user, name: 'Grace' };
items = [...items, newItem];
filters = { ...filters, active: true };
```

This is why OnPush pushes you toward **immutable updates** — which is also easier to reason about and debug. In the Angular Lab **Change Detection** lesson, press *Mutate name* then *Replace user* and watch the two cards react differently.

4 · SIGNALS AND ONPUSH

Why the two work so well together

When a template reads a signal, Angular records **which component** depends on it. When the signal changes, exactly that component is marked for checking — even if it has **no inputs** and sits deep inside an OnPush tree.

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `Items in cart: {{ cart.count() }}`,
})
export class BadgeComponent {
  readonly cart = inject(CartService); // count is a signal
}
```

The result: fine-grained updates with no reference juggling. Angular also supports **zoneless** change detection (`provideZonelessChangeDetection()`), which drops Zone.js and relies on signals, events and `markForCheck()` to know when to update.

5 · THE TOOLS

ChangeDetectorRef and friends

TOOL	WHAT IT DOES	WHEN TO USE
<code>markForCheck()</code>	Marks this component and its ancestors dirty; the next normal check will include it	Data changed outside Angular's knowledge (a callback, a plain field)
<code>detectChanges()</code>	Checks this component's view right now	Rare: after manual DOM-affecting work, or in tests
<code>detach()</code> / <code>reattach()</code>	Removes / restores the view from checks	Special cases like a frozen, very expensive view
<code>NgZone.runOutsideAngular()</code>	Runs code without triggering a check afterwards	High-frequency events (scroll, mousemove, animation frames)
<code>async</code> pipe	Subscribes, marks dirty on each emission, unsubscribes for you	Showing an Observable in a template

6 · MAKING IT FASTER

Practical rules

- Use **OnPush** by default for new components, especially list rows and "dumb" display components.
- Keep template expressions cheap. A method call like `{{ expensive() }}` runs on every check. Use a `computed()` signal or a pure pipe instead.
- Track list items. `@for (item of items; track item.id)` lets Angular reuse DOM nodes instead of rebuilding the list.
- Update immutably so OnPush components notice.
- Move noisy events outside Angular with `runOutsideAngular` and re-enter only when the UI must change.
- Measure first. Use the Angular DevTools *Profiler* tab to see which components are checked and how long each takes.

7 · THE FAMOUS ERROR

ExpressionChangedAfterItHasBeenCheckedError (NG0100)

In development mode Angular checks every binding **twice** to make sure nothing changed during the check. If a binding's value is different the second time, you see this error.

```
// Bad: the template reads a value that changes while it is being checked
get randomLabel() { return Math.random(); }           // different every call
// Bad: a child changes data the parent already displayed
ngAfterViewInit() { this.parentTitle = 'Updated'; }
```

Fix: make bindings pure and stable, move the change into a signal / `computed()`, or update state earlier (for example in the constructor or an event handler) rather than in a lifecycle hook that runs after the view was checked.

8 · COMMON MISTAKES

Things that trip people up

- "My OnPush component does not update." You mutated the input. Replace the object/array instead.
- Calling `detectChanges()` everywhere to force the UI. Prefer signals or `markForCheck()`.
- Heavy getters / methods in templates that run on every check.
- Subscribing manually and assigning to a field in an OnPush component, with no `markForCheck()`. Use the `async` pipe or `toSignal()`.
- Expecting OnPush to stop *child events*. A click inside the component (or its children) always marks it dirty.

9 · CHEAT SHEET & QUICK QUIZ

SITUATION	ANSWER
Input object mutated	OnPush will not see it → pass a new object
Signal read in the template changes	That component is re-checked
Observable in the template	Use <code>async</code> pipe (or <code>toSignal</code>)
Scroll / mousemove handlers	<code>runOutsideAngular</code>
Slow template	Profile first; use <code>computed()</code> , <code>track</code> and OnPush

Q: When is an OnPush component re-checked?

New input reference, an event inside it, an async pipe emission, a signal it reads changing, or `markForCheck()`.

Q: Why does mutating an input not update an OnPush child?

OnPush compares by reference; the same object means "no change".

Q: Why do signals pair well with OnPush?

The read tells Angular exactly which component to re-check.

Try it live: youssefezzat17.github.io/angular-lab/change-detection